Indicode Front-End Status Report - v0.9.8 (pre-release)

== Indicode Compiler Front-End Status Report

Front-end package version: 0.9.8 (pre-release; not a complete compiler).

Status date: 2026-11-03 Scope: Indicode front end only; LLVM/backend/runtime work is excluded.

The companion catalogue audit PDF contains the full 40-item checklist and remaining-work detail; it is generated from FRONT_END_REMAINING_WORK.md.

-- Executive summary

The front end reads Indicode source, tokenizes and parses the supported language, checks names and types, reports source-aware diagnostics, and lowers valid input to a typed HIR that is independently verified.

Measure	Current status
Weighted front-end readiness estimate	97.25% (rounded to 97%)
Front-end catalogue items implemented and tested at their stated scope	40 / 40
Partially implemented catalogue items	0 / 40
Not started catalogue items	0 / 40
Current CTest suite	23 / 23 passed

The 97.25% figure is a judgment-based, weighted estimate of front-end readiness, including release and conformance gaps. It is not the percentage of the 40-item catalogue: the code audit found an implementation and focused test for all 40 items at their stated API or v1 acceptance level. The count does not mean all APIs are integrated into source syntax, that every possible language program is supported, or that the complete compiler is finished.

-- Scope and limitations

The working boundary is a documented Indicode v1 front end that can produce verified typed HIR. The accepted syntax and static rules are specified in LANGUAGE_SPEC_V1.md and SEMANTICS_V1.md. Package version 0.9.8 and grammar version 1.0.0 are separate version numbers.

The v1 parser remains single-file and does not accept module/import or overload syntax. Public resolution APIs provide dotted module path lookup, manifest-based imports, alias/visibility/cycle checks, and exact-signature overload selection; they are not yet wired into parsing, semantic analysis, or the CLI. Generic function declarations, constraint checks, type substitution, effect checking, and the documented limited ownership/borrowing checks are implemented in the parser and semantic analyzer. Incremental parsing is implemented at the source-edit level with revision-based revalidation and clean reparse matching for repaired or invalid fragments. The source-aware symbol/reference index is available as an HIR snapshot API; editor-protocol integration remains. These boundaries remain visible rather than being claimed as language-level support. There is no LLVM adapter, machine-code generation, executable runtime, tensor execution, or device backend. The project is not production-ready: full conformance, coverage-guided fuzzing, sanitizer gates, release CI, and compatibility/ABI policy remain.

-- Weighted progress by acceptance area

Area	Weight	Done	Points	Evidence and remaining gap
Project/build foundation	5%	95%	4.75	C++20/CMake library, CLI, CTest, install/export package and separate package consumer; release CI remains.
Source manager and diagnostics	10%	97%	9.70	Owned source buffers, indexed line tables, byte/scalar columns, tab-aware diagnostics, notes, fix-its and diagnostic JSON schema v1; terminal display-width and editor integration remain.
Lexer	15%	92%	13.80	Typed token payloads, trivia, UTF-8 validation, strings, radix integers, separators and nested comments; Unicode identifiers, suffix policy and complete numeric grammar remain.

Parser and AST	15%	99%	14.85	Grammar v1, precedence, recovery, golden printer, exhaustive visitor and ownership/lifetime contract; broader conformance remains.
Name resolution	10%	96%	9.60	v1 scopes/namespaces and stable identities, plus tested module/import and exact-signature overload resolution APIs; grammar/analyzer integration remains.
Type checking and semantic validation	15%	99%	14.85	v1 declarations, generic functions/constraints, effects, limited ownership modes, expressions, calls, conversions, records, collections, tensor shapes, constants and conservative flow/return checks; full language and runtime semantics remain outside scope.
Typed HIR and verification	15%	99%	14.85	AST lowering, stable symbols, AST/HIR JSON schema v2 and independent checks of types, scopes, identities, calls, ownership/effect metadata, mutability and return paths; deserialization and broader compatibility guarantees remain.
CLI and diagnostics tooling	5%	99%	4.95	check, tokens, ast, ast-json, typed, typed-json, source excerpts, diagnostic JSON schema v1, and HIR symbol/reference index; editor-protocol integration and terminal display-width support remain.
Tests, docs and integration	10%	99%	9.90	23 passing CTest cases including AST scalability, symbol index, module/import/overload APIs, malformed-input smoke tests and installed-package consumer; full conformance, fuzzing, sanitizers and CI remain.
Total	100%		97.25%	Rounded display: 97%.

Calculation: 4.75 + 9.70 + 13.80 + 14.85 + 9.60 + 14.85 + 14.85 + 4.95 + 9.90 = 97.25.

-- What is implemented

-- Source model and lexer

- Owned source files, stable source IDs, spans, indexed CR/LF/CRLF line lookup, byte offsets, UTF-8 scalar columns, and four-column tab stops.
- Stable lexer diagnostics and recovery for malformed tokens, UTF-8, numeric forms, strings, and comments.
- Typed keyword/integer/float/string payloads; original spelling and source spans are retained.
- Source-spanned whitespace and line/block-comment trivia; nested block comments preserve newline separators.
- ASCII identifiers, supported string escapes and Unicode scalar escapes, decimal floats, binary/octal/hex integers, and digit separators.

-- Grammar, parser and AST

- Normative grammar version 1.0.0, recursive-descent declarations/statements, and Pratt expression parsing with documented precedence/associativity.
- Functions, records, typed fields, lists, calls with named arguments, indexing, member access, loops, integer ranges, conditionals and nested else if.
- Declaration, statement and delimited-expression recovery; valid later statements and sibling expressions are retained after malformed input.
- Deterministic AST printer golden output; public AstVisitor callbacks for every declaration, statement kind and expression kind; full-tree walk_ast.
- Documented AST ownership/lifetime contract. Program owns syntax nodes; nodes do not borrow token/source text. Source contents must still be retained separately when source excerpts are needed.
- AST JSON schema version 2 and source spans.

-- Name binding, types and semantic checks

- Separate v1 value/function/type lookup rules, lexical block scopes, permitted

outer-value shadowing, duplicate declaration diagnostics, and stable identities for values, functions, record types/fields, builtins and references.
- Type inference/checking for supported declarations, literals, expressions, operators, function calls, named arguments, records, lists, text, indexing, integer ranges, generic function constraints/substitution, and checked numeric conversions.
- Static tensor type validation for numeric elements and exact rank, dimensions, and element types. Symbolic dimensions match by name; tensor operations/broadcasting/device placement are not supported.
- Bounded integer/Boolean constant evaluation with short-circuiting, signed-64-bit overflow and division-by-zero checks, and a deterministic 256-node evaluation budget.
- Condition/count/return checks, unreachable-statement diagnostics, and conservative definite-return analysis. A loop is not assumed to execute.
- Effect inference for I/O, allocation, mutation and transitive calls; limited ownership modes with move/use-after-move and call-scoped borrow checks.
- Semantic rules are tested and independently rechecked in typed HIR where applicable.

-- Diagnostics, HIR, CLI and packaging

- Text diagnostics with source excerpts/carets, stable severity/code identifiers, notes/secondary spans, fix-its and multi-error reporting.
- Diagnostic JSON schema v1 with version envelope, locations, notes, fix-it data, escaping, deterministic ordering and golden-output tests.
- Project-owned typed HIR schema version 2 and independent verification of types, symbols, visibility, mutability, calls, loops, returns, ownership modes, and effect summaries.
- CLI commands: check, tokens, ast, ast-json, typed, and typed-json; JSON output is schema-versioned.
- CMake install/export package and a separately built package-consumer test.
- HIR-backed symbol/reference indexing with lexical scope lookup and full-snapshot invalidation by source revision; this is a library API, not an LSP server.
- Public module/import resolver APIs with deterministic path lookup, alias and public-export resolution, missing/ambiguous diagnostics, and cycle rejection.
- Exact-type overload candidate selection with named-argument matching, no-match reporting, and ambiguity reporting; parser/semantic integration is not part of v1 syntax.
- A 10,000-statement AST parse/destruction acceptance test; the documented unique_ptr ownership design remains in place rather than adding an arena.
- Effect checking and ownership validation are in place and verified by the semantic regression suite.
- Incremental parsing now supports edit ranges and revision-based fragment invalidation, including malformed and repaired fragment recovery, and verifies results against clean full parses.

-- Catalogue audit: all 40 front-end items

The original front-end catalogue covers IDs 445–484. “Implemented and tested” means the code and focused tests meet the stated v1 or public-API acceptance level; API-only items and their integration limits are identified explicitly.

Status	IDs	Concepts
Implemented and tested (40)	445–484	Code and focused-test evidence covers all catalogue items at their stated v1 or public-API acceptance level. Resolver and index APIs have the integration boundaries described above; this count is not a claim of full-language or complete-compiler readiness.

There are no catalogue items currently marked “Partial” or “Not started”. The front-end catalogue is complete by item count for this audit.
Weighted front-end readiness remains 97.25%, and source-language integration and production/release readiness remain incomplete.

-- Remaining work to reach the next milestones

1. Language integration: decide whether module/import/overload syntax enters
a future grammar version, then connect the resolver APIs to parsing, semantic analysis, HIR identities, and CLI workflows (IDs 468–470).
2. Production/release readiness: add requirement-linked full v1
conformance, coverage-guided fuzzing, sanitizers, clean offline CI/release builds, and a documented compatibility policy. These are acceptance
gaps reflected in the weighted areas even when not separate catalogue IDs.

-- Verification recorded for this report

- A fresh Release CMake configure/build succeeds; the full CTest suite passes:
23/23.
- Covered tests include lexer/parser/AST/visitor/semantics/HIR/diagnostics,
AST large-input destruction, symbol definitions/references/scopes/rebuild invalidation, module/import/overload resolver APIs, incremental parser
revalidation for malformed and repaired fragments, CLI JSON, malformed-input robustness, and installed-package consumption.
- AST and typed-HIR JSON emit schema version 2; diagnostic JSON emits schema
version 1.

-- Related specifications

- Language syntax v1
- Semantic rules v1
- AST ownership and lifetime
- Parser recovery v1
- Diagnostic schema v1
- Detailed catalogue checklist
- Detailed catalogue checklist PDF